CODE :

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];

void* philosopher (void* num) {

   int id= *(int*)num;

   while(1) {

      printf("Philosopher %d thinking\n", id);

      sleep(1);

      pthread_mutex_lock(&forks [id]);

      pthread_mutex_lock(&forks[(id+1)%N]);

      printf("Philosopher %d eating\n", id);

      sleep(1);

      pthread_mutex_unlock(&forks[id]);

      pthread_mutex_unlock(&forks[(id+1)%N]);

   }

}

   int main() {

      pthread_t p[N];

      int i, ids [N];

      for(i=0; i < N; i++)
```

```
    pthread_mutex_init(&forks[i], NULL);
    for(i=0; i < N; i++){
        ids[i] = i;
        pthread_create(&p[i], NULL, philosopher, &ids[i]);
    }
    for(i=0; i < N; i++)
    pthread_join(p[i], NULL);
    return 0;
}
```

OUTPUT :

```
Output
Philosopher 0 thinking
Philosopher 1 thinking
Philosopher 3 thinking
Philosopher 2 thinking
Philosopher 4 thinking
Philosopher 0 eating
Philosopher 3 eating
Philosopher 0 thinking
Philosopher 3 thinking
Philosopher 4 eating
Philosopher 2 eating
Philosopher 4 thinking
Philosopher 2 thinking
Philosopher 1 eating
Philosopher 3 eating
Philosopher 3 thinking
Philosopher 1 thinking
Philosopher 0 eating
Philosopher 2 eating
```